

Öğrenme fazları: **1 • Ön-öğretim** **2 • İçerik** **3 • Katılım** **4 • Değerlendirme** **5 • İzleme**

1 • Ön-öğretim **Ön Bilgi:** C'den taşıdıklarınız: main, tipler (int, double, char), if/for/while, işaretçi (*/&) ve ayrı derleme fikri. Python'dan tanıdık gelen: değişken, döngü, fonksiyon çağırma sezgisi. Bu hafta aynı temeli daha sıkı tip güvenliği, iostream ve std::string ile C++ diline taşırsınız.

» Derleme ve Bağlama Modeli

Aşamalar

Her .cpp = bir **çeviri birimi**: önışlemci (#include genişletir) → derleyici → .o nesne dosyası. Bağlayıcı (linker) tüm .o'ları birleştirip çalıştırılabilir üretir.

Header'ın Rolü

Yalnız bildirim taşır; önışlemci aşamasında ilgili çeviri birimine kopyalanır (fabrika-montaj hattı: header = teknik kroki, .cpp = ayrı ayrı üretilen parça).

C++ Zihniyeti

Aynı derle+bağla modeli, daha sıkı sözler: tip güvenliği (C'nin void*/örtük dönüşüm serbestliğine kıyasla), aşırı yükleme, referanslar. RAIİ'ye ilk bakış: kaynağın ömrü nesnenin ömrüyle eşleşir (mekanizma Hafta 4'te).

» C'den C++'a: Mini Karşılaştırma

printf("%d", x);	std::cout << x; — format string hatası riski yok
scanf("%d", &x);	std::cin >> x; — hedef tipi cin bilir
malloc/free	new/delete — RAIİ ile ömür nesneye bağlanır (Hafta 4)
char* dizi	std::string — birleştirme +, uzunluk .size()
tek dosya derleme	.hpp/.cpp + CMake — çok dosyalı proje

```
1 // C hali // C++ hali
2 #include <stdio.h> #include <iostream>
3 int main(void){ int main(){
4     printf("Merhaba!"); std::cout << "Merhaba!";
5     return 0;} return 0;}
```

» namespace, iostream, string

namespace

İsim çakışmasını çözen isimlendirme kapsayıcısı (soyadı/şehir kodu gibi). std ad alanı: std::cout. using namespace std; header'da KAÇINILIR (tercih: std:: öneki ya da seçici using std::cout;).

cin/cout Akışları

<< veriyi hedefe gönderir, >> girdiyi kaynaktan değişkene teslim alır (posta kutusu, ok yönü = akış yönü).

```
1 std::string isim;
2 std::cout << "Adınız? ";
3 std::cin >> isim;
4 std::cout << "Merhaba, " << isim << "!\n";
```

std::string Tadimlığı

char* yerine: birleştirme +, uzunluk .size() — manuel bellek yönetimi yok.

```
1 std::string ad = "Ada", soyad = "Lovelace";
2 std::string tam = ad + " " + soyad;
3 std::cout << tam << " (" << tam.size() << ")";
```

» Referans (int&) vs İşaretçi

Referans

Var olan değişkene ikinci isim (lakap): boş olamaz, kurulduktan sonra başka değişkene yeniden bağlanamaz; çağırın tarafta */& gerekmez.

```
1 void ikiyeKatla(int& sayi) { sayi = sayi * 2; }
2 int n = 21;
3 ikiyeKatla(n); // n = 42, & yazılmaz
```

İşaretçi (Karşılaştırma İçin)

Bir adres tutan ayrı değişken (yazılı ev adresi): nullptr olabilir, başka adrese yeniden yönlendirilebilir; okumak için *p (dereference) gerekir.

Karmaşıklık (big-O)

Referans parametre, kopya oluşturmadan orijinal nesneye erişir: büyük std::string/nesnelerde değere göre geçişteki $O(n)$ kopya maliyetini $O(1)$ 'e indirir.

» auto, const, Aralık Tabanlı for

auto — Tip Çıkarımı

Derleyici, sağ taraftan değişkenin tipini kendisi belirler (otomatik vites: sürücü rotayı seçer, vites tipini araç seçer). Uzun/karmaşık tip adlarında kullanışlı; okunabilirliği gizleyecekse dikkatli kullanılır.

```
1 auto a = 7; // int
2 auto toplam = a + 35; // int
```

const Referans Parametre

Duvara çakılı tabela: değer değiştirilemez. const std::string& iki faydayı birleştirir: kopyalama maliyeti yok + değiştirmeme garantisi.

```
1 void mesajYazdir(const std::string& mesaj) {
2     std::cout << mesaj << '\n';
3 }
```

Aralık Tabanlı for

İndeks yazmadan eleman gezer; C'nin klasik indeksli for'una kıyasla daha az hata yüzeyi.

```
1 int sayilar[] = {10, 20, 30, 40};
2 for (int sayi : sayilar) {
3     std::cout << sayi << '\n';
4 }
```

» Sık Yapılan Hatalar

Ortak Hata / Tuzak

- using namespace std; header'a (.hpp) yazmak: onu include eden HER dosyaya isim kirliliği yayılır.
- Referansı boş bırakmaya çalışmak (int& r; ilklemeden): referans kurulurken bağlanmak ZORUNDADIR, C'deki NULL işaretçi mantığıyla karıştırılmaz.
- #pragma once unutmak: aynı header iki yerden include edilirse içerik çeviri birimine iki kez kopyalanır → yeniden tanımlama hatası.
- Büyük nesneyi değere göre almak (std::string s): gereksiz kopya; parametre değişmeyecekse const std::string& tercih edin.

Ne Zaman Hangisi?

- Referans vs işaretçi: parametre HER ZAMAN geçerli bir nesneyse referans; boş olabilir/yeniden atanacaksa işaretçi.
- auto** vs açık tip: tip uzun/karmaşıksa (ileride yineleyiciler) **auto**; okunabilirlik önemliyse açık tip.
- Değere göre vs **const&**: küçük tip (**int**, **double**) değere göre; büyük/kopyalanması pahalı tip (**std::string**, sınıf) **const&**.

» Kütüphaneyi Kurmak: .hpp/.cpp + namespace

Başlık/Kaynak Ayrımı

.hpp: bildirim, başında **#pragma once** (çoklu include koruması — etkinlikte tek seferlik giriş damgası). .cpp: tanım/ gövde — yalnız bir kez derlenir.

```
1 // nokta.hpp
2 #pragma once
3 namespace geometri {
4 struct Nokta { double x, y; };
5 Nokta topla(const Nokta& a, const Nokta& b);
6 void yazdir(const Nokta& n);
7 }
```

Kullanım (nokta_demo.cpp)

```
1 #include "nokta.hpp"
2 geometri::Nokta a{1.0, 2.0}, b{3.0, 4.0};
3 auto toplam = geometri::topla(a, b);
4 geometri::yazdir(toplam); // (4, 6)
```

Bu iskelet (header/source + namespace + CMake), Hafta 3'te **Kitap** sınıfıyla aynı desende tekrar kullanılır.

» Derleme Komutları

g++ -std=c++20 -c nokta.cpp	Kaynağı nesne dosyasına derler
g++ nokta.o nokta_demo.o -o demo	Nesne dosyalarını bağlar
clang++ -std=c++20 nokta_demo.cpp nokta.cpp -o demo	Tek adımda derle+bağla
cmake -S . -B build	CMake ile derleme klasörü oluşturur
cmake --build build	CMakeLists.txt'e göre projeyi derler

CMakeLists.txt İskeleti

```
1 cmake_minimum_required(VERSION 3.20)
2 project(hafta01_temel CXX)
3 set(CMAKE_CXX_STANDARD 20)
4 add_executable(nokta_demo nokta_demo.cpp nokta.cpp)
```

Birden fazla .cpp tek hedefte birleşir; elle **g++** komutu yazmadan derleme+bağlama adımlarını yönetir.

» Neden Böyle?

■ Tanım: RAII (İlk Bakış)

Kaynağın ömrü nesnenin ömrüyle eşleşir — otel oda kartı gibi: kart alındığında oda size aittir, kartiade edince oda otomatik boşalır. Mekanizma (kurucu/yıkıcı ile) Hafta 4'te işlenecek; bu hafta yalnız isim + tek cümlelik tanımım.

💡 İpucu

g++ ile **clang++** ikisi de C++ derleyicisidir, aynı komut satırı bayraklarıyla (bkz. komut tablosu) çalışır — biri varsa diğerini kullanabilirsiniz. CMake bu seçimi proje genelinde soyutlar, tek tek komut yazmaz.

✍ Kendinizi Sınavın 1

- Header neden yalnız bildirim taşır? *Önişlemci onu her çeviri birimine kopyalar; tanım tekrarlanırsa yenden tanımlama hatası olur.*
- cin >> sayi;** ile **scanf("%d", &sayi);** farkı? *cin hedef tipi bilir, format string hatası riski yok.*
- const std::string&** parametre iki fayda: *kopya maliyeti yok + değiştirmeme garantisi.*

📖 Özet

Bu hafta C++'ın derle+bağla modelini, **namespace/iostream/** referansları, **auto/const**'u ve **.hpp/.cpp+CMake** iskeletini gördünüz. **Hafta 2:** fonksiyonları derinleştirip (aşırı yükleme, referans/işaretçi parametre) C++'ın bellek modelini — **stack/heap**, **new/delete**, RAII'nin ilk somut örneğini — işleyeceksiniz.

5 • **İzleme** Nerede Kullanılır: Yeni bir C++ dosyasına başlarken **iostream** ile girdi/çıkı, **std::string** ile metin işleyin; büyük nesneyi fonksiyona geçirirken **const&** ile kopyayı önleyin, çağıranda değeri değiştirecekseniz düz referans (**int&**) alın. Birden çok .cpp'yi .hpp + namespace ile ayırıp CMake ile derleyin. Küçük tipte (**int**, **double**) değere göre, uzun/karmaşık tipte **auto** tercih edin.